MongoDB notes by RAVISHASTRY

MongoDB is a NoSQL, document-oriented database used to store data in flexible, JSON-like documents. These notes cover the basics through commonly used commands, with examples you can practice.

1. What is MongoDB?

* MongoDB is a NoSQL database.

* It stores data in BSON (Binary JSON) format.

* It uses collections instead of tables and documents instead of rows.

* It supports flexible schemas.

* It is commonly used with Node.js, Express.js, and React.

SQL vs MongoDB

SQL database	MongoDB
Database	Database
Table	Collection
Row	Document
Column	Field
Primary key	_id
JOIN	$lookup (aggregation)

Example MongoDB document:

javascript

```
{
  name: "Ravishastry",
  age: 32,
  stream: "Electrical Engineering",
  totalPercentage: 85.0
}
```

2. Important MongoDB terms

* Database: Container for collections.

* Collection: Group of documents, similar to a SQL table.

* Document: A single record containing fields and values.

* Field: A key such as name, age, or address.

* _id: Unique identifier for each document, automatically generated if omitted.

3. Basic MongoDB commands

These commands work in the MongoDB Shell (mongosh).

Database commands

javascript

// Show all databases

show dbs

// Create or switch to a database

use college

// Display the current database

db

// Delete the current database

db.dropDatabase()

Note: A database generally appears in show dbs after data has been stored in it.

Collection commands

javascript

```
// Show collections
show collections

// Create a collection
db.createCollection("students")

// Delete a collection
db.students.drop()
```

4. CRUD operations

CRUD stands for Create, Read, Update, and Delete.

A. Create — insert documents

Insert one student:

javascript

```
db.students.insertOne({
  name: "Ravishastry",
  age: 32,
  stream: "Electrincal Engineering",
```

```
  totalPercentage: 85.0
})
```

Insert multiple students:

javascript

```
db.students.insertMany([
  {
    name: "Mrunal Thakur",
    age: 33,
    stream: "Information Technology",
    totalPercentage: 91.2
  },
  {
    name: "Alia Bhat",
    age: 28,
    stream: "Mechanical Engineering",
    totalPercentage: 76.8
  },
  {
    name: "Mamitha",
    age: 21,
    stream: "Civil Engineering",
    totalPercentage: 88.4
  }
])
```

B. Read — find documents

javascript

```
// Find all students
db.students.find()

// Find one student
db.students.findOne({ name: "Ravishastry" })

// Find students with age 21
db.students.find({ age: 21 })

// Find students in Computer Science
db.students.find({ stream: "Computer Science" })

// Display selected fields
db.students.find(
  {},
  { name: 1, stream: 1, _id: 0 }
)
```

C. Update — modify documents

javascript

```
// Update one student
db.students.updateOne(
  { name: "Ravishastry" },
  { $set: { age: 33 } }
)

// Update multiple students
db.students.updateMany(
  { stream: "Computer Science" },
  { $set: { department: "CSE" } }
)

// Increase percentage by 2
db.students.updateOne(
  { name: "Ravishastry" },
  { $inc: { totalPercentage: 2 } }
)
```

D. Delete — remove documents

javascript

```
// Delete one student
db.students.deleteOne({ name: "Ravishastry" })

// Delete all students in a particular stream
db.students.deleteMany({
  stream: "Electrical Engineering"
})

// Delete all documents in the collection
db.students.deleteMany({})
```

Important: deleteMany({}) removes every document in the collection. Use it carefully.

5. MongoDB comparison operators

Operator	Meaning	Example
$eq	Equal to	{ age: { $eq: 21 } }
$ne	Not equal to	{ age: { $ne: 21 } }
$gt	Greater than	{ age: { $gt: 20 } }
$gte	Greater than or equal	{ age: { $gte: 21 } }
$lt	Less than	{ age: { $lt: 25 } }
$lte	Less than or equal	{ age: { $lte: 22 } }
$in	Matches any listed value	{ age: { $in: [20, 21] } }
$nin	Matches none of the listed values	{ age: { $nin: [20, 21] } }

Examples:

javascript

```
// Students scoring above 80%
db.students.find({
  totalPercentage: { $gt: 80 }
})

// Students aged between 20 and 22
db.students.find({
  age: { $gte: 20, $lte: 22 }
})

// Students from selected streams
db.students.find({
  stream: {
    $in: ["Computer Science", "Civil Engineering"]
  }
})
```

6. Logical operators

Operator	Purpose
$and	All conditions must match
$or	At least one condition must match

$not	Negates a condition
$nor	None of the conditions should match

javascript

```
// AND: both conditions
db.students.find({
  $and: [
    { age: { $gte: 20 } },
    { totalPercentage: { $gt: 80 } }
  ]
})

// OR: either condition
db.students.find({
  $or: [
    { stream: "Computer Science" },
    { totalPercentage: { $gt: 90 } }
  ]
})
```

8. Projection

Projection selects which fields to return.

javascript

```
// Show only name and percentage
```

```
db.students.find(
  {},
  { name: 1, totalPercentage: 1, _id: 0 }
)

// Hide age
db.students.find(
  {},
  { age: 0 }
)
```

You generally cannot mix inclusion and exclusion in the same projection, except for _id.

9. Important update operators

Operator	Use
$set	Add or change a field
$unset	Remove a field
$inc	Increase or decrease a numeric value
$push	Add an item to an array
$pull	Remove matching array items
$addToSet	Add an item only if it is not already present
$rename	Rename a field

Examples:

javascript

```
// Add or change a field
db.students.updateOne(
  { name: "Priya" },
  { $set: { city: "Mumbai" } }
)

// Remove a field
db.students.updateOne(
  { name: "Priya" },
  { $unset: { city: "" } }
)

// Increase age by 1
db.students.updateOne(
  { name: "Priya" },
  { $inc: { age: 1 } }
)
```

10. Aggregation pipeline

Aggregation is used to filter, group, sort, and calculate information from documents.

Common stages:

* $match — filter documents.

* $group — group documents and calculate values.

* $sort — sort results.

* $project — select or reshape fields.

* $limit — limit the number of results.

* $skip — skip results.

* $unwind — create a separate pipeline document for each array item.

* $lookup — combine related data from another collection.

Example 1: Find students scoring above 80%

javascript

```
db.students.aggregate([
  {
    $match: {
      totalPercentage: { $gt: 80 }
    }
  }
])
```

Example 2: Calculate average percentage

javascript

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      averagePercentage: {
        $avg: "$totalPercentage"
      }
```

```
    }
  }
])
```

Example 3: Count students in each stream

```
javascript
db.students.aggregate([
  {
    $group: {
      _id: "$stream",
      totalStudents: { $sum: 1 }
    }
  },
  {
    $sort: { totalStudents: -1 }
  }
])
```

11. Indexes

An index helps MongoDB find documents more efficiently.

```
javascript
// Create an index on name
db.students.createIndex({ name: 1 })
```

```
// Create a descending index on percentage
db.students.createIndex({ totalPercentage: -1 })

// View indexes
db.students.getIndexes()

// Remove a specific index
db.students.dropIndex({ name: 1 })
```

Indexes can speed up queries, but they also consume storage and add overhead to writes.

12. Relationships in MongoDB

MongoDB supports embedded documents and references.

Embedded document:

```
javascript
{
  name: "Ravishastry",
  address: {
    city: "Bengaluru",
    state: "Karnataka"
  }
}
```

Query an embedded field:

javascript

```
db.students.find({
  "address.city": "Bengaluru"
})
```

Array field:

javascript

```
{
  name: "Mrunal Thakur",
  skills: ["JavaScript", "React", "Node.js"]
}
```

Query an array element:

javascript

```
db.students.find({
  skills: "React"
})
```

13. Common MongoDB data types

* String — “R”ravishastry
* Integer — 21
* Double — 85.6
* Boolean — true
* Array — ["Java", "Python"]

* Object — { city: "Mumbai" }

* ObjectId — ObjectId("...")

* Date — ISODate("2026-01-01T00:00:00Z")

* Null — null

15. Frequently asked interview questions

16. What is MongoDB?

17. What is the difference between SQL and NoSQL?

18. What are collections and documents?

19. What is the purpose of the _id field?

20. What is the difference between find() and findOne()?

21. What is the difference between updateOne() and updateMany()?

22. What is the difference between $set and $inc?

23. What is an aggregation pipeline?

24. What is an index, and why is it useful?

25. What is the difference between embedding and referencing?

26. What is the difference between deleteOne() and deleteMany()?

27. What are $match, $group, and $lookup?

28. What is BSON?

29. What is the difference between insertOne() and insertMany()?

30. How does pagination work using skip() and limit()?

Practice these MongoDB tasks

1. Insert 10 engineering students into a collection.

2. Find students whose percentage is greater than 85.

3. Sort students from highest to lowest percentage.

4. Update a student's address.

5. Count the number of students in each stream.

6. Find the top 5 students by percentage.

MongoDB main interview concepts

Database and collection basics → CRUD → query operators → sorting and pagination → aggregation → indexes → Node.js integration.